```
# Lab 10: Visualizing Flow Data

**Description:**
- Each row represents a flow from a source to a target with a value.
- Used for Sankey, alluvial, chord, directed, and stream graph visualizations.
- 6 rows, 3 columns: `source`, `target`, `value`.

**Features used:**
- Source, target, value (flow quantity)

**Observation:**
- The dataset is ideal for demonstrating flow visualizations between stages or entities.
```

1. Sankey Diagram

- Visualizes flow between two or more stages.
- Node thickness reflects flow quantity.

```
import pandas as pd
import plotly.graph_objects as go
import plotly.express as px
import holoviews as hv
import networkx as nx
import matplotlib.pyplot as plt
import altair as alt
```

```
df = pd.read_csv('data_sankey.csv')
df = df.rename(columns=lambda x: x.strip().lower())

# Use the correct column names
labels = list(pd.unique(df[['prior_disorder', 'later_disorder']].values.ravel('K')))
label_to_index = {label: i for i, label in enumerate(labels)}
source_indices = df['prior_disorder'].map(label_to_index)
target_indices = df['later_disorder'].map(label_to_index)

fig = go.Figure(go.Sankey(
    node=dict(label=labels, pad=15, thickness=20),
    link=dict(source=source_indices, target=target_indices, value=df['n'])
))
fig.update_layout(title_text="Sankey Diagram", font_size=12)
fig.show()
```

Observation:

- The thickest links represent the largest flows, making it easy to spot major transitions.

2. Alluvial Plot (Parallel Categories)

Description:
This plot shows how cases flow from one disorder to another. The width and color of each band represent the number of cases (`n`) transitioning from a prior disorder to a later disorder.

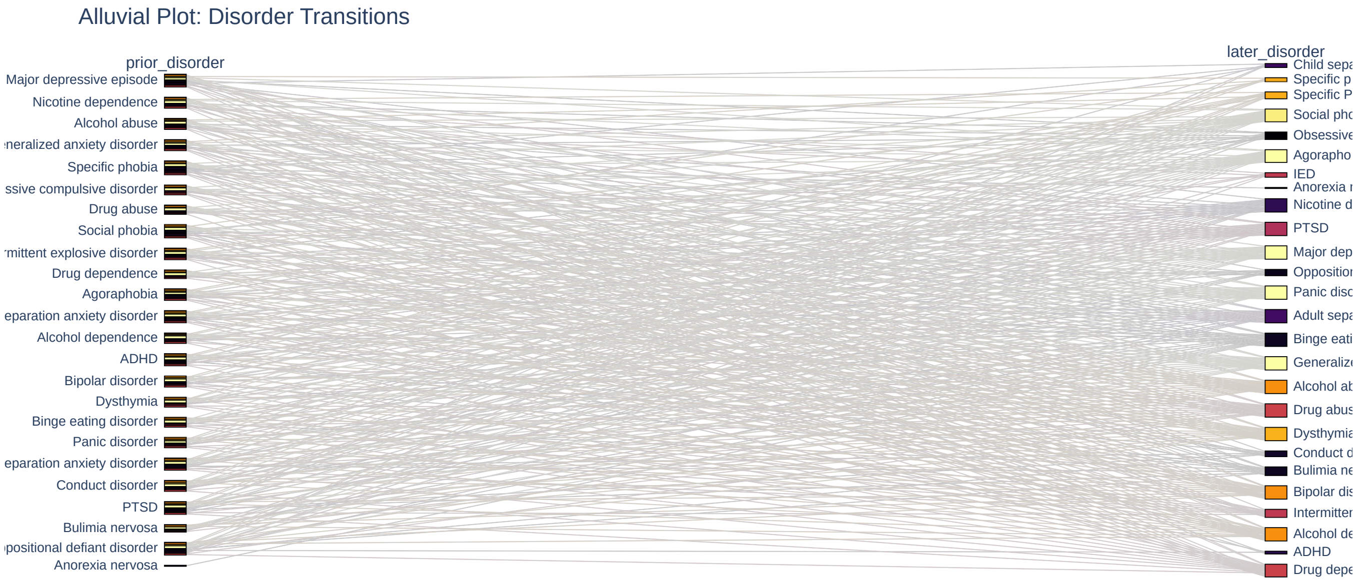
Features used:

- `prior_disorder` (source)
- `later_disorder` (target)
- `n` (flow value)

Observation:

- The thickest bands highlight the most common transitions between disorders.

```
fig = px.parallel_categories(
    df,
    dimensions=['prior_disorder', 'later_disorder'],
    color="n",
    color_continuous_scale=px.colors.sequential.Inferno
)
fig.update_layout(title="Alluvial Plot: Disorder Transitions")
fig.show()
```



3. Chord Diagram

Description:
A chord diagram visualizes the bidirectional relationships between disorders. Each arc connects two disorders, and its thickness represents the number of cases (`n`) transitioning between them.

Features used:

- `prior_disorder` (source)
- `later_disorder` (target)
- `n` (connection strength)

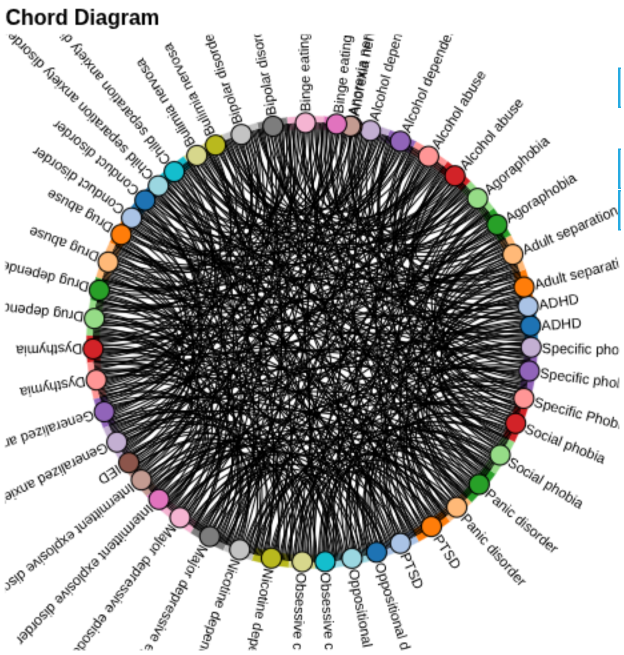
Observation:

- The most prominent arcs indicate the strongest transitions between specific disorders.

```
# Install holoviews and bokeh if not already installed
!pip install holoviews bokeh --quiet
```

```
from holoviews import opts
hv.extension('bokeh')

# Prepare data for chord diagram
links = list(df[['prior_disorder', 'later_disorder', 'n']].itertuples(index=False, name=None))
chord = hv.Chord(links).opts(
    opts.Chord(
        cmap='Category20', edge_color='source', labels='index',
        node_color='index', width=500, height=500, title="Chord Diagram"
    )
)
chord
```



4. Directed Graph (Flow Style)

Description:

This directed graph represents transitions between disorders as a network. Arrows show the direction of flow, and edge thickness reflects the number of cases (n).

Features used:

- `prior_disorder` (source)
- `later_disorder` (target)
- `n` (edge weight)

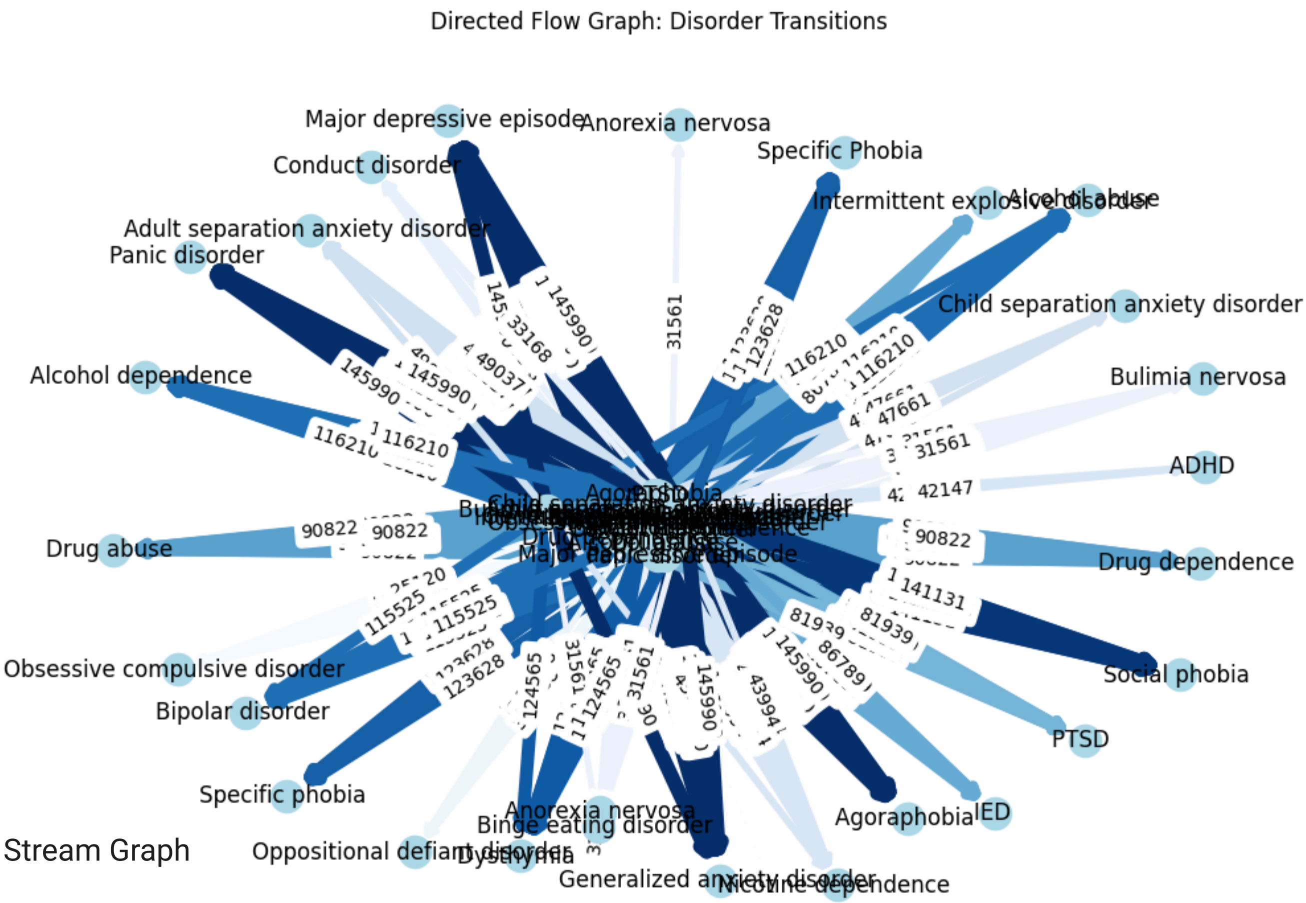
Observation:

- The graph structure highlights the main transition pathways and their relative strengths.

```
G = nx.DiGraph()
for _, row in df.iterrows():
    G.add_edge(row['prior_disorder'], row['later_disorder'], weight=row['n'])

pos = nx.spring_layout(G, seed=42)
weights = [G[u][v]['weight'] for u, v in G.edges()]
max_w = max(weights)
widths = [2 + 6*(w/max_w) for w in weights]

plt.figure(figsize=(10,7))
nx.draw(G, pos, with_labels=True, node_color='lightblue', arrows=True, width=widths, edge_color=weights, edge_cmap=plt.cm.Bl
nx.draw_networkx_edge_labels(G, pos, edge_labels={(u, v): f"{d['weight']}" for u, v, d in G.edges(data=True)})
plt.title("Directed Flow Graph: Disorder Transitions")
plt.show()
```



5. Stream Graph

Description:
A stream graph visualizes how the number of transitions from each prior disorder changes across different later disorders (or, if you have a time/stage column, use that for the x-axis).

Features used:

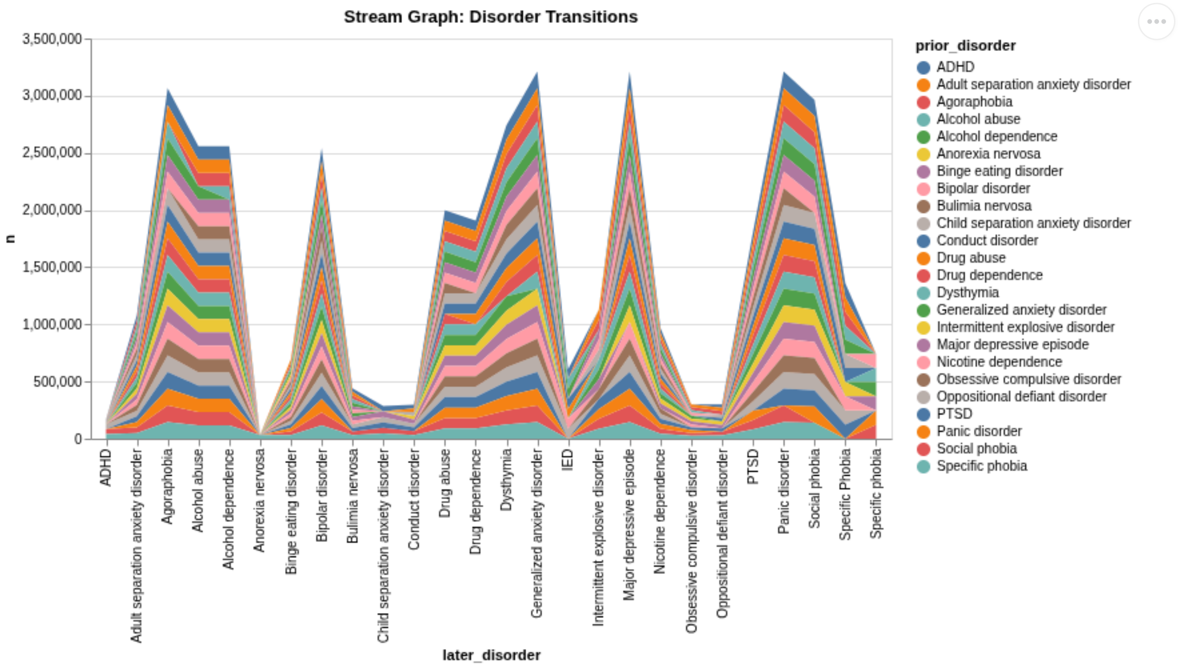
- `prior_disorder` (category/color)
- `later_disorder` (x-axis)
- `n` (y-axis/height)

Observation:

- The stream graph shows which prior disorders contribute most to each later disorder.

```
# Aggregate for stream graph: sum n for each prior-later pair
stream_data = df.groupby(['later_disorder', 'prior_disorder']).agg({'n': 'sum'}).reset_index()

chart = alt.Chart(stream_data).mark_area().encode(
    x='later_disorder:N',
    y='n:Q',
    color='prior_disorder:N'
).properties(
    width=600,
    height=300,
    title="Stream Graph: Disorder Transitions"
)
chart.show()
```


Stream Graph: Disorder Transitions
n
later_disorder
prior_disorder
ADHD
Adult separation anxiety disorder
Agoraphobia
Alcohol abuse
Alcohol dependence
Anorexia nervosa
Binge eating disorder
Bipolar disorder
Bulimia nervosa
Child separation anxiety disorder
Conduct disorder
Drug abuse
Drug dependence
Dysthymia
Generalized anxiety disorder
Intermittent explosive disorder
Major depressive episode
Nicotine dependence
Obsessive compulsive disorder
Oppositional defiant disorder
PTSD
Panic disorder
Social phobia
Specific phobia
IED
Specific Phobia
3,500,000
3,000,000
2,500,000
2,000,000
1,500,000
1,000,000
500,000
0